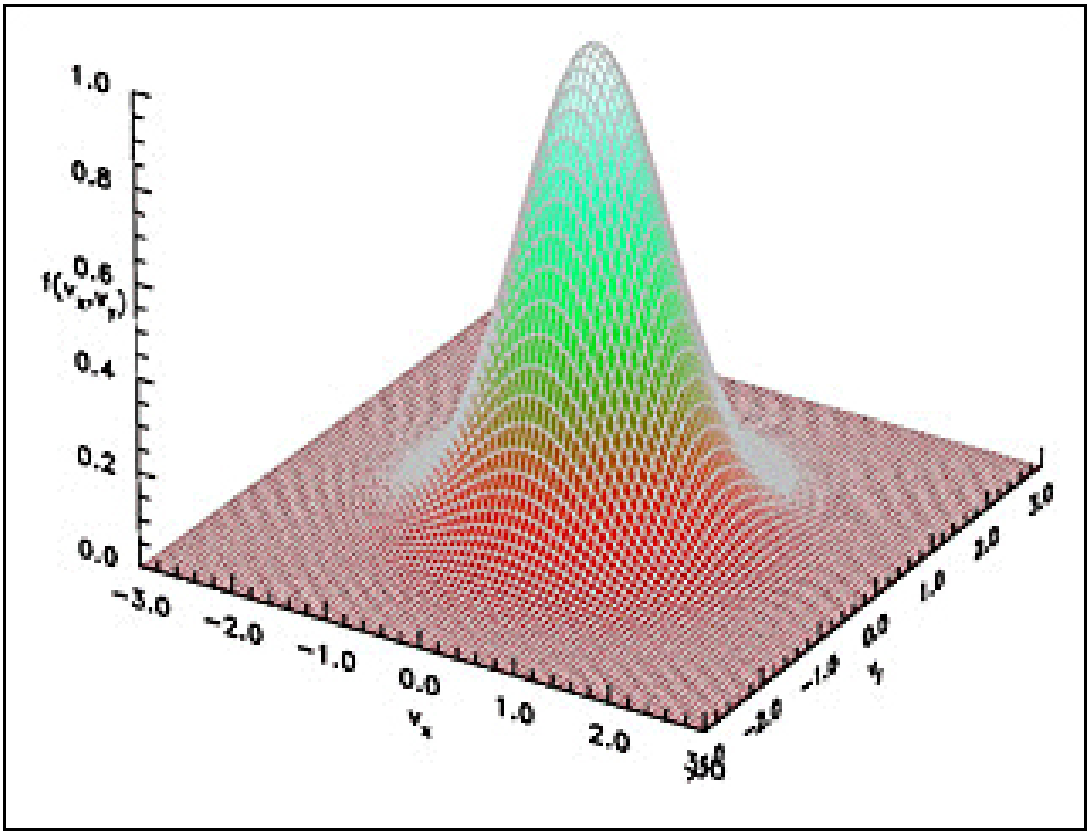
CSIE5123: Fundamental of Artificial Intelligence

Vinsong

June 8, 2026

Abstract

The lecture note of 2026 Spring Fundamental of Artificial Intelligence by professor 陳尚澤、陳縕儂.



Contents

Chapter 1

Floating-point systems

Lecture 1

2025.12.09

1.1 Floating-point basics

This chapter is mainly based on the science of floating-point arithmetics which is based on the IEEE standard 754.

> **Remark.** The floating-point number system here introduction is to help us redesign a better floating-point system while we are doing deep learning implementation.

1.1.1 Why learning floating-point operations?

> **Example.** A one-variable problem
>
> $$\min_x f(x) \quad \text{where } x \geq 0$$

In the normal program, we should set an upper bound of x, or x may be wrongly increased to ∞. We have to find the largest representable number in the computer. Or how to define the "infinity" in the computer?

> **Example.** A ten-variable problem
>
> $$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{where } x_i \geq 0, i = 1, 2, \ldots, 10$$

We want to know how many are zeros, we may use

```
for (int i = 0; i < 10; i++)
    if (x[i] == 0) count++;
    // count the number of zeros
```

But people say that don't do the comparison of floating-point due to the precision issue.

```
double epsilon = 1.0e-12;
for (int i = 0; i < 10; i++)
    if (x[i] <= epsilon) count++;
```

Which is better? How to chose "`epsilon`"? Can't do the comparison of floating-point? We need to understand the floating-point representation.

1.1.2 Floating-point Format

We know `float` (single precision): 4 bytes and `double` (double precision) 8 bytes in C/C++.

Note. We usually spare the bits for deeplearning system to save memory and speed up the training process.

Definition 1.1.1 (format). A floating-point system requires

- a base β
- precision p
- significand (mantissa) $d.d\dots d$

Example.

$$0.1 = 1.00 \times 10^{-1} \qquad (\beta = 10, p = 3)$$
$$\approx 1.1001 \times 2^{-4} \qquad (\beta = 2, p = 5)$$

exponent: -1 and -4; significand: 1.00 and 1.1001.

We let $e_{\max}$ to be the largest exponent and $e_{\min}$ to be the smallest exponent. Then, β^p is the possible significand, and $e_{\max} - e_{\min} + 1$ is the possible exponent.

$$\lceil \log_2(e_{\max} - e_{\min} + 1) \rceil + \lceil \log_2(\beta^p) \rceil + 1$$

bits for storing a floating-point number. 1 bit for the sign.

In the pratical it is more complicated.

Definition 1.1.2 (Normalized floating-point number). A normalized floating-point number is a number whose significand is in the form of $1.d_1 d_2 \dots d_{p-1}$.

Example. 0.1 is not; 1.00×2^{-3} is

Now the issue will become represent of zero. We natually use $1.0 \times \beta^{e_{\min}-1}$ to represent zero, but it is not allow in computer system.

Remark. We usually reserve some special numbers to represent 0, ∞, and NaN.

1.1.3 Relative error and Ulps

Example. When $\beta = 10, p = 3$, for 3.14159

We represent it as 3.14×10^0, and the error is $|3.14159 - 3.14| = 0.00159 = 0.159 \times 10^{-2}$ i.e.

- 10^{-2} is the unit of the last place.
- `ulps`: 0.159 is the unit in the last place

The absolute error is 0.159 ulps.

Definition 1.1.3 (ulps). The unit in the last place (ulps) is the unit value of the least significant digit in a floating-point number.

We also use "relative error" to measure the error. The relative error is $0.00159/3.14159 \approx 0.0005$.
For a number $d.d\ldots d \times \beta^e$, the largest error cause by rounding is

$$0.\underbrace{0\ldots0}_{p-1}\beta' \times \beta^e \quad \beta' = \beta/2$$

So, Error $= \frac{\beta}{2} \times \beta^{-p} \times \beta^e$ and

$$1 \times \beta^e \leq \text{original number} < \beta \times \beta^e$$

The relative error will between

$$\frac{\frac{\beta}{2} \times \beta^{-p} \times \beta^e}{\beta^e} \text{ and } \frac{\frac{\beta}{2} \times \beta^{-p} \times \beta^e}{\beta^{e+1}}$$

So,

$$\text{relative error} \leq \frac{\beta}{2} \times \beta^{-p} \tag{1.1.1}$$

Definition 1.1.4 (machine epsilon). The machine epsilon is the upper bound of the relative error of a floating-point number $\epsilon = \frac{\beta}{2} \times \beta^{-p} = \beta^{1-p}/2$

Example. $x = 12.35 \Rightarrow \tilde{x} = 1.24 \times 10^1 \quad (p = 3,\ \beta = 10)$

- 1 ulps $= 0.01 \times 10^1 = 0.1$, $\epsilon = 1/2 \cdot 10^{-2} = 0.005$
- error $= 0.05 = 0.005 \times 10^1 = 0.5$ ulps
- relative error $= 0.05/12.35 \approx 0.004 = 0.8\epsilon$
- $8x = 98.8,\ 8\tilde{x} = 9.92 \times 10^1$ error $= 4.0$ ulps, relative error $= 0.04 = 0.8\epsilon$

Note. ulps and ϵ is used interchangeably usually.

1.2 Guard digits

In some situation, the order of floating-point computation and rounding may not have a huge effect on the final result. So, usually we choose to

$$\boxed{\text{Round}} \quad \Rightarrow \quad \boxed{\text{Compute}}$$

to let the computation to be more efficient. But in some cases, the error may be huge.

Example.

$$x = 10.1$$
$$y = 9.93$$
$$x - y = 0.17$$

- Round and then compute:

$$10.1 - 9.93 = 1.01 \times 10^1 - 0.99 \times 10^1 = 0.02 \times 10^1 = 2.00 \times 10^{-1}$$

  - error $= 2.00 \times 10^{-1} - 0.17 = 0.03$
  - ulps $= 0.01 \times 10^{-1} = 10^{-3}$
  - error $= 30$ ulps
  - relative error

$$= \frac{0.03}{0.17} = \frac{3}{17}$$

- Compute and then round:

$$10.1 - 9.93 = 0.17 \approx 1.7 \times 10^{-1}$$

  - error $= 1.7 \times 10^{-1} - 0.17 = 0$

So, we have to know how big will the error be if we choose to round first then compute.

Theorem 1.2.1. Using p precision with base β for $x - y$, the relative error can be as large as $\beta - 1$

Proof. Let

$$x = 1.0\ldots0,\ y = 0.\underbrace{\eta\ldots\eta}_{p \text{ digits}},\quad \eta = \beta - 1$$

The correct solution is $x - y = \beta^{-p}$. Under some rounding scheme, we have the solution

$$1.0\ldots0 - 0.\underbrace{\eta\ldots\eta}_{p-1 \text{ digits}} = \beta^{-p+1}$$

> **Note.** Above is a bad rounding just ignore the last η

Relative error is

$$\frac{|\beta^{-p+1} - \beta^{-p}|}{\beta^{-p}} = \beta - 1$$

■

Example. ($p = 3$, $\beta = 10$) $x = 1.00,\ y = 0.999,\ x - y = 0.001 = 10^{-3}$

The compute solution is $1.00 \times 10^0 - 0.99 \times 10^0 = 0.01 \times 10^0 = 0.01$. The relative error is

$$\frac{|0.01 - 0.001|}{0.001} = 9$$

Such a huge error occurs when the two numbers are close to each other. So, we need to use guard digits to store the intermediate result to avoid such a huge error.

Definition 1.2.1 (guard digits). Guard digits are extra digits used in the intermediate steps of a calculation to reduce the effect of rounding errors and improve the accuracy of the final result.

In here, the guard digits is p increased by 1 in the device for addition and subtraction

For the above example, we have the computation become

$$1.010 \times 10^1 - 0.993 \times 10^1 = 0.017 \times 10^1 = 0.17$$

note that $0.17 = 1.70 \times 10^{-1}$ can be stored in the system with $p = 3$ and $\beta = 10$. The relative error becomes 0.

Note. We only allocate more digit precision for the calculation device (i.e. the substraction process) to store the intermediate result.

Example. ($p = 3$, $\beta = 10$) $x = 110,\ y = 8.59, x - y = 101.41$

$$\begin{aligned} 110 - 8.59 &= 1.100 \times 10^2 - 0.085 \times 10^2 \\ &= 1.015 \times 10^2 \approx 1.01 \times 10^2 \end{aligned}$$

Relative error is

$$\frac{|1.01 \times 10^2 - 101.41|}{101.41} \approx 0.003$$

$\epsilon = \frac{1}{2} \cdot 10^{-2} = 0.005$.

Theorem 1.2.2. Using $p+1$ digits for $x - y \ \Rightarrow$ relative rounding error $< 2\epsilon$ (ϵ: machine epsilon)

Proof. We first make some assumption

- $x > y$
- $x = x_0 \,.\, x_1 x_2 \ldots x_{p-1} \times \beta^0$, the proof will be similar for not β^0

For the first two condition,

1. If $y = y_0 \,.\, y_1 y_2 \ldots y_{p-1}$ no error
2. If $y = 0.y_1 y_2 \ldots y_p \ \Rightarrow$ 1 guard digit, exact $x - y$ rounded to a closest number $\Rightarrow$ error $< \epsilon$

In general, $y = 0.0 \ldots 0\ y_{k+1} \ldots y_{k+p}$, we let $\bar{y}$: truncate y to $p+1$ digits.

$$|y - \bar{y}| < (\beta - 1)(\beta^{-p-1} + \beta^{-p-2} + \cdots + \beta^{-p-k}) \tag{1.2.1}$$

After the truncation, we have to compute

$$x - \bar{y}$$

Then we rounded the result get

$$x - \bar{y} + \delta$$

Thus

$$\text{error} \le 0.\ \underbrace{0 \ldots 0}_{p-1 \text{ digits}}\ (\beta/2) \quad \text{(worst case)}$$

Therefore,

$$|\delta| \le (\beta/2) \cdot \beta^{-p} \tag{1.2.2}$$

Then the error is

$$(x - y) - (x - \bar{y} + \delta) = \bar{y} - y - \delta$$

Now, we consider three cases:

1° Case 1: $x - y \geq 1$, from (1.2.1) and (1.2.2), we have the relative error

$$\begin{aligned}\frac{|\bar{y} - y - \delta|}{x - y} &\leq \frac{|\bar{y} - y - \delta|}{1} \\ &\leq \beta^{-p}[(\beta - 1)(\beta^{-1} + \beta^{-2} + \cdots + \beta^{-k}) + \beta/2] \\ &= \beta^{-p}[(\beta - 1)\beta^{-k}(1 + \cdots + \beta^{k-1}) + \beta/2] \\ &= \beta^{-p}[(\beta - 1)\beta^{-k} \cdot \frac{\beta^k - 1}{\beta - 1} + \beta/2] \\ &= \beta^{-p}[(1 - \beta^{-k}) + \beta/2] \\ &< \beta^{-p}[1 + \beta/2] \leq 2\epsilon\end{aligned}$$

2° Case 2: $x - \bar{y} \leq 1$, enough digits to store $x - \bar{y}$, so the $\delta = 0$, and the relative error is

$$\frac{|\bar{y} - y|}{x - y}$$

The smallest $x - y$ is

$$1.0 - 0.0 \ldots 0\ \rho \ldots \rho > (\beta - 1)(\beta^{-1} + \beta^{-2} + \cdots + \beta^{-k})$$

we have k zeros, p ρ and $\rho = \beta - 1$, from (1.2.1), we have the relative error

$$\begin{aligned}&\leq \frac{|\bar{y} - y|}{(\beta - 1)(\beta^{-1} + \beta^{-2} + \cdots + \beta^{-k})} \\ &< \frac{(\beta - 1)\beta^{-p}(\beta^{-1} + \beta^{-2} + \cdots + \beta^{-k})}{(\beta - 1)(\beta^{-1} + \beta^{-2} + \cdots + \beta^{-k})} = \beta^{-p} = 2\epsilon\end{aligned}$$

3° Case 3: $x - y < 1$ but $x - \bar{y} > 1$
If $x - \bar{y} = 1.\underbrace{0 \ldots 1}_{p} \Rightarrow x - y \geq 1$: a contradiction.

Because

$$|y - \bar{y}| < \beta^{-p} = 0.\underbrace{0 \ldots 1}_{p}$$

then

$$x - y = \underbrace{(1 + \beta^{-p})}_{x - \bar{y}} - \underbrace{|y - \bar{y}|}_{< \beta^{-p}} > 1$$

Proof is completed. ■

Conclusion: we can add "some" (not only 1) guard digits to make the relative error small enough. Especially when the two numbers are close to each other. To add one bit on the adder is very cheap.

1.3 Cancelation

Cancelatin is an important source of numerical instability. It occurs when we subtract two nearly equal numbers.

1.3.1 Catastrophic Cancelation

Example. b = 3.34, a = 1.22, c = 2.28, calculat $b^2 - 4ac$

- True value: $b^2 - 4ac = 0.0292$
- Calculated value: $b^2 \approx 11.2,\ 4ac \approx 11.1$, so $b^2 - 4ac \approx 0.1$

The Error $= 0.1 - 0.0292 = 0.0708$

- Computed answer $= 0.1 = 1.00 \times 10^{-1}$
- ulps $= 0.01 \times 10^{-1} = 10^{-3}$
- $0.0708 \approx 70$ ulps

The error is huge when we subtract two nearly equal numbers. This is called **catastrophic cancelation**. We can use **benign cancelation** to avoid this problem. For example, we use guard digits to get the relative error being small.

1.3.2 Benign Cancelation

However, guard digits can only reduce the error of operations between two exact number. In this case, b^2 and $4ac$ already contain errors. We can use **reformulation** to avoid this problem. For example,

Example.

$$\frac{-b - \sqrt{b^2 - 4ac}}{2a} \tag{1.3.1}$$

If $b^2 \gg 4ac$, there is no problem for $\sqrt{b^2 - 4ac} \approx |b|$, but $-b + \sqrt{b^2 - 4ac}$ will cause catastrophic cancelation if $b > 0$.
We can reformulate by multiplying $-b - \sqrt{b^2 - 4ac}$ if $b > 0$.

$$\frac{2c}{-b + \sqrt{b^2 - 4ac}} \tag{1.3.2}$$

Now we can use (1.3.1) if $b < 0$ and (1.3.2) if $b > 0$ to avoid catastrophic cancelation.

Example. Calculate $x^2 - y^2$.

Assume $x \approx y$, $(x - y)(x + y)$ is much more accurate than $x^2 - y^2$ because x^2 and y^2 maybe rounded, so $x^2 - y^2$ may cause catastrophic cancelation. But, $x - y$ can be calculated by guard digits.

Remark. It is difficult to avoid all catastrophic cancelation but possible to remove some. A catastrophic cancellation is replaced by a benign cancellation.

Example. Calculate

$$A = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2} \tag{1.3.3}$$

a, b, c are the lengths of the triangle.

If $a \approx b + c$,

$$s = \frac{a+b+c}{2} \approx a$$

and $s - a$ may cause catastrophic cancelation. A new formula is by Kahan[1986]: for $a \geq b \geq c$,

$$A = \frac{1}{4}\sqrt{(a+(b+c))(c-(a-b))(c+(a-b))(a+(b-c))} \tag{1.3.4}$$

Remark. Reformulation is a powerful tool to avoid catastrophic cancelation. It only works if guard digit is used.

1.3.3 Exactly Rounded Operations

Usually we choose to

$$\boxed{\text{Round}} \quad \Rightarrow \quad \boxed{\text{Compute}}$$

However, it may be not accurate. So, we introduce **exactly rounded** operations, which is to compute exactly then rounded to the nearest.

Definition (definition of rounding). There are two types of rounding:

Definition 1.3.1 (Rounding up). When the result's last digit $\geq$ half of the base, we round up, otherwise, we round down.

Definition 1.3.2 (Rounding even). Round to the closest value with even least significant digit.

Example. Round up

- $11.5 \Rightarrow 12$
- $12.5 \Rightarrow 13$

Round even

- $11.5 \Rightarrow 12$
- $12.5 \Rightarrow 12$

Theorem 1.3.1 (Reiser and Knuth). Let

$$x_0 = x, \quad x_1 = (x_0 \ominus y) \oplus y, \quad \cdots, \quad x_n = (x_{n-1} \ominus y) \oplus y$$

If $\oplus, \ominus$ is an exactly rounded using even rounding, then

$$x_n = x \ \forall n \text{ or } x_n = x_1 \ \forall n \geq 1$$

Example. $\beta = 10,\ p = 3,\ x = 1.00,\ y = -0.555$

$x - y = 1.555,\ x \ominus y = 1.56,\ (x \ominus y) + y = 1.56 - 0.555 = 1.005$

- Rounding up:

$$x_1 = (x \ominus y) \oplus y = 1.01, \quad x_1 - y = 1.565 \Rightarrow 1.57, \quad (x_1 \ominus y) + y = 1.57 - 0.555 = 1.015$$

$$x_2 = (x_1 \ominus y) \oplus y = 1.02$$

  Increase until $x_n = 9.45$.

- Rounding even:

$$x_1 = (x \ominus y) \oplus y = 1.00, \quad x_1 - y = 1.555 \Rightarrow 1.56, \quad (x_1 \ominus y) + y = 1.56 - 0.555 = 1.005$$

$$x_2 = (x_1 \ominus y) \oplus y = 1.00$$

Note. For the implementation, an array of words or floating-points used too much memory. Goldberg [1990] proposed a method to use only **two guard digits and one sticky bit**, the result is exact same as exactly rounded operations.

1.4 IEEE 754 Standard

1.4.1 Basics

IEEE 754 is the most used standard for floating-point arithmetic in computers nowadays. There are two of floating-point standard

- 754: specify $\beta = 2, p = 24$ for single precision, $\beta = 2, p = 53$ for double precision.
- 854: $\beta = 2$ or 10

Note. In standard 854, it accept $2, 10$ because

- $\beta = 2$ smaller β causes smaller relative error.
- $\beta = 10$ is most used.

Consider

$$\beta = 16, p = 1 \text{ versus } \beta = 2, p = 4$$

which both used 4 bits to store the significand, but there ϵ is different

$$\epsilon_{16} = \frac{16}{2} \cdot 16^{-1} = \frac{1}{2}, \quad \epsilon_2 = \frac{2}{2} \cdot 2^{-4} = \frac{1}{16}$$

However, $\beta = 16$ is used by IBM/370 due to two reasons:

1° Assume 4 bytes = 32 bits, $\beta = 16, p = 6$
significand:

$$4 \times 6 = 24 \text{ bits}$$

exponent:

$$32 - 24 - 1 = 7 \text{ bits (1 bit for sign)}$$

The range of exponent is $16^{-2^6} \to 16^{2^6} = 2^{2^8}$.
If we instead use $\beta = 2$ and to keep the same range of exponent, then

$$9 \text{ bits } (-2^8 \to 2^8) \text{ for exponent}$$

and

$$32 - 9 - 1 = 22 \text{ bits for significand}$$

Same exponents, less significand for $\beta = 2$ (24 vs. 22)

2° The cost of shifting: If $\beta = 16$, less frequently to adjust exponents when adding or subtracting two numbers. But nowadays the cost of shifting is not important since the computation speed.

1.4.2 IEEE standard: Significand and Exponent

For single precision, $\beta = 2, p = 24$ (23 bits as normalized), exponent 8 bits, and 1 bit for sign. Thus,

$$32 = 1 \text{ (sign bit) } + 8 \text{ (exponent) } + 23 \text{ (normalized significand)}$$

Example. $176.625 = 1.0101100101 \times 2^7$

$$0 \qquad 10000110 \qquad 01011001010000000000000$$

1 of $1.\cdots$ is not stored since the normalized.

Note. Here we use the bias of exponent

$$10000110 = 128 + 4 + 2 = 134,\ 134 - 127 = 7$$

Note that exponent may be negative, but here we don't use a sign bit for exponents

In this standard, it use rounding even

Binary	rounded	reason
10.00011	10.00	($< 1/2$, down)
10.00110	10.01	($> 1/2$, up)
10.11100	11.00	($1/2$, up)
10.10100	10.10	($1/2$, down)

Here is the summary of all standard:

IEEE	Fortran	C	Bits	Exp.	Mantissa
Single	REAL*4	float	32	8	24
Single-extended			44	≤ 11	32
Double	REAL*8	double	64	11	53
Double-extended	REAL*10	long double	$\geq$80	≥ 15	$\geq$64

Observe the single extended

$$44 \neq 11 \text{ (bits for exponent) } + 32 \text{ (bits for significand)}$$

Hardware implementation of extended precision normal don't use a hidden bit.
Minimal normalized positive number

$$1 \times 2^{-126} \approx 1.17 \times 10^{-38}$$

$e_{\min} = -126$

8 bits for exponent: 0 to 255. IEEE uses a biased approach for exponents

$$(0 \text{ to } 255) - 127 = -127 \text{ to } 128$$

There are some cases for exponent:

- -127: denormalized numbers, and 0
- -126 to 127: exponent for normalized numbers
- 128: special quantity (NaN, ∞)

Thus,

$$e_{\min} = -126, \quad e_{\max} = 127$$

> **Note.** The reason for not using $e_{\min} = -127$ is that $1/2^{e_{\min}}$ not overflow, $1/2^{e_{\max}}$ underflow.

1.4.3 IEEE standard: Extended Precision

Some may use more digits for intermediate calculations, which is called **extended precision**. For example, binary-decimal conversion, or some calculators use 13 digits for intermediate calculations but round to 10 digits for display.
Another example is that writing 9 digits is enough for short. Though $10^8 > 2^24$, 8 digits are not enough. From Section 2.1.2 of Goldberg [1990], in reading the 9-digit number, if extended precision is available, an efficient algorithm can be done so that the original binary representation is recovered.

1.4.4 IEEE standard: Exactly Rounded Operations

IEEE standard requires results of addition, subtraction, multiplication and division exactly rounded. The reason of doing so is to leave the portability of software.

Also, square root, remainder, conversion between integer and floating-point, internal formats and decimal are exactly rounded.

IEEE does not require transcendental functions to be exactly rounded (e.g. $\sin, \cos, \exp$). They cannot specify the precision because they are arbitrarily long

1.4.5 IEEE standard: Special Quantities

On some computer, (e.g. IBM 370) there is no special quantity, the value like $\sqrt{-4} = 2$ and it print a error message. But IEEE standard has special quantities

`NaN` (Not a Number)

which means not every pattern of bits is a valid number. Here is the summary of all cases:

Exponent	significand	represents
$e = e_{\min} - 1$	$f = 0$	$+0, -0$
$e = e_{\min} - 1$	$f \neq 0$	$0.f \times 2^{e_{\min}}$
$e_{\min} \leq e \leq e_{\max}$		$1.f \times 2^e$
$e = e_{\max} + 1$	$f = 0$	$\pm\infty$
$e = e_{\max} + 1$	$f \neq 0$	NaN

Sometimes even 0/0 occurs, the program can continue, which is better than stop and print an error message.

Example. Finding $f(x) = 0$, try different x ' s, even $0/0$ happens, other values may be ok.

If $b^2 - 4ac < 0$,

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

return NaN, and $-b \pm \sqrt{b^2 - 4ac}$ return NaN as well.

Remark. In general when a NaN is in an operation, result is NaN

Here are some examples of producing NaN:

Operation	NaN by
$+$	$\infty + (-\infty)$
$\times$	$0 \times \infty$
$/$	$0/0, \infty/\infty$
REM	x REM 0, ∞ REM y
$\sqrt{}$	$\sqrt{x}$ when $x < 0$

1.4.6 IEEE standard: Infinity

$\beta = 10,\ p = 3,\ e_{\max} = 98,\ x = 3 \times 10^{70}$, x^2 overflow and replaced by ∞. Note that

$$0/0 = \text{NaN},\ 1/0 = \infty,\ -1/0 = -\infty \ \Rightarrow \text{ nonzero divided by 0 is } \infty \text{ or } -\infty$$

In the computaion, we replace ∞ with x, let $x \to \infty$.

Example.

$$\frac{x}{x^2+1} \text{ vs } \frac{1}{x + x^{-1}}$$

For $\dfrac{x}{x^2+1}$, if x is large, x^2 overflows, so $\dfrac{x}{\infty} = 0$ but not $\dfrac{1}{x}$.

For $\dfrac{1}{x + x^{-1}}$, if x is large, $\frac{1}{x}$ is ok, which is better.

When for $x = 0$,

$$\frac{1}{x + x^{-1}} = \frac{1}{0 + \infty} = \frac{1}{\infty} = 0$$

is also better.

Note. If no infinity arithmetic, an extra instruction is needed to test if $x = 0$. This may interrupt the pipeline.

1.4.7 IEEE standard: Signed Zero

To discuss why we need signed zero, first, it is available with 1 sign bit. If no signed zero, $1/(1/x)$ will fails when $x = \pm\infty$

$$x = \infty,\ 1/x = 0,\ 1/0 = +\infty$$
$$x = -\infty,\ 1/x = 0,\ 1/0 = +\infty$$

Definition 1.4.1 (Comparison of signed zero). In IEEE standard, $+0 = -0$.

± 0 is useful for some case

Example. We can define

$$\log x \equiv \begin{cases} -\infty & x = 0 \\ \texttt{NaN} & x < 0 \end{cases}$$

Definition 1.4.2 (Underflow). Underflow means a value smaller than the smallest floating-point number occurs.

When face a small underflow negative number, $\log x$ should return NaN. However, in the definition above,

$$\boxed{x \text{ underflow negative}} \quad \Rightarrow \quad \boxed{x \overset{\text{round}}{=} 0} \quad \Rightarrow \quad \boxed{\log x = -\infty}$$

With signed zero, we can define $\log x$ as

$$\log x \equiv \begin{cases} -\infty & x = +0 \\ \texttt{NaN} & x = -0 \\ \texttt{NaN} & x < 0 \end{cases}$$

Positive underflow will round to $+0$, and negative underflow will round to -0.

Example (Complex arithmetic).

$$\frac{1}{\sqrt{z}} \quad \text{and} \quad \sqrt{\frac{1}{z}}$$

If $z = -1$,

$$\frac{1}{\sqrt{-1}} = \frac{1}{i} = -i \quad \sqrt{\frac{1}{-1}} = i$$

Thus, $1/\sqrt{z} \neq \sqrt{1/z}$. Which happen because

$$i^2 = (-i)^2 = -1$$

However, by some restrictions (or ways of calculation), they can be equal.
If $z = -1 = -1 + 0i$,

$$\frac{1}{-1+0i} = \frac{-1-0i}{(-1)^2 + (0)^2} = -1 - 0i$$

then

$$\sqrt{\frac{1}{z}} = \sqrt{-1-0i} = -i$$

with signed zero, we can let $\sqrt{1/z} = 1/\sqrt{z}$.
But there is a disadvantage of signed zero, if $x = +0$ and $y = -0$, then

$$\frac{1}{x} = \infty, \quad \frac{1}{y} = -\infty$$

1.4.8 IEEE standard: Denormalized Numbers

Example. Consider

$$\beta = 10,\ p = 3,\ e_{\min} = -98,\ x = 6.87 \times 10^{-97},\ y = 6.81 \times 10^{-97}$$

x, y are ok but $x - y$ underflow and round to 0 even though $x \neq y$.

Note. It is important to preserve the property of

$$x = y \quad \Leftrightarrow \quad x - y = 0$$

with this case: `if (x != y) { z = 1/(x-y); }` The statement is true, but z becomes ∞.

IEEE use denormalized numbers to solve this problem, which is the most controversial part of IEEE standard. If denormalized number is used, 0.6×10^{-98} is also a floating-point number.

We do not store the leading 1 for $1.\cdots$. Recall the vaild range of exponent, $e \geq e_{\min}$, and we have $1.d\ldots d \times 2^e$. For denormalized numbers, we allow $e = e_{\min} - 1$, and the corresponding value is

$$0.d\ldots d \times 2^{e_{\min}}$$

Remark. The reason why not using

$$1.d\ldots d \times 2^{e_{\min}-1}$$

is that then we cannot represent

$$0.0d\ldots d \times 2^{e_{\min}}$$

which is even smaller than the smallest normalized number.

Example. $6.87 \times 10^{-97} - 6.81 \times 10^{-97} = 0.06 \times 10^{-97}$

is underflow due to the cancelation.

Here is an example of using denormalized numbers:

Example.

$$\frac{a+bi}{c+di} = \frac{(a+bi)(c-di)}{(c+di)(c-di)} = \frac{ac+bd}{c^2+d^2} + \frac{bc-ad}{c^2+d^2}i$$

If c or $d > \sqrt{\beta}\beta^{e_{\max}/2}$, there will be overflow.

Definition 1.4.3 (Overflow). Overflow means a value larger than the largest floating-point number occurs.

Lemma 1.4.1 (Smith's lemma).

$$\frac{a+bi}{c+di} = \begin{cases} \frac{a+b(d/c)}{c+d(d/c)} + \frac{b-a(d/c)}{c+d(d/c)}i & \text{if } (|d| < |c|) \\ \frac{b+a(c/d)}{d+c(c/d)} + \frac{-a+b(c/d)}{d+c(c/d)}i & \text{if } (|d| \geq |c|) \end{cases}$$

This is a reformulation to avoid overflow. However, using Smith ' s formula, some issues may occur without denormalized numbers:

If

$$a = 2 \times 10^{-98},\ b = 1 \times 10^{-98},\ c = 4 \times 10^{-98},\ d = 2 \times 10^{-98}$$

then

$$\frac{d}{c} = 0.5, \quad c + d \cdot \frac{d}{c} = 5 \times 10^{-98}$$

$$b \cdot \frac{d}{c} = 1 \times 10^{-98} \times 0.5 = \mathbf{0}, \quad a + b \cdot \frac{d}{c} = 2 \times 10^{-98}$$

Solution $= 0.4$, which is wrong.

If denormalized numbers are used, 0.5×10^{-98} can be stored

$$a + b \cdot \frac{d}{c} = 2 \times 10^{-98} + 0.5 \times 10^{-98} = 2.5 \times 10^{-98}$$

Solution $= 0.5$, which is correct.

Note. Usually hardware does not support denormalized numbers directly, we use software to simulate it.

Note. Program may be slow when there is a lot of underflow.

1.5 Trap Handlers

1.5.1 Exception, Flags, Trap handlers

When we face with some special situations (e.g. divide by zero), we have to know what happens. For this purpose, IEEE requires vendors to provide a way to get status flags. **overflow**, **underflow**, **division by zero**, **invalid operation**, **inexact**.

Definition (IEEE exception)**.** IEEE defines five exceptions:

Definition 1.5.1 (Overflow)**.** Larger than the maximal floating-point number.

Definition 1.5.2 (Underflow)**.** Smaller than the minimal floating-point number.

Definition 1.5.3 (Division by zero)**.** Nonzero divided by zero.

Definition 1.5.4 (Invalid operation)**.** $\infty + (-\infty)$, $0 \times \infty$, $0/0$,, ∞/∞, x REM 0, ∞ REM y, $\sqrt{x}, x < 0$, any comparison involves a `NaN`.

Remark. Invalid returns `NaN`; But `NaN` may not be from invalid operations.

Definition 1.5.5 (Inexact)**.** The result is not exact: $\beta = 10, p = 3, 3.5 \times 4.2 = 14.7$ exact, $3.5 \times 4.3 = 15.05 \Rightarrow 15.0$ not exact.

Remark. Inexact exception is raised so often, usually we ignore it.

Here is the summary of all exceptions and their corresponding handlers:

Exception	when trap disabled	argument to handler
overflow	$\pm\infty$ or $\pm 1.1\cdots 1 \times 2^{e_{\max}}$	round$(x \times 2^{-\alpha})$
underflow	$0, \pm 2^{e_{\min}}$, or denormalized	round$(x \times 2^{\alpha})$
division by zero	∞	operands
invalid	NaN	operands
inexact	round(x)	round(x)

Definition 1.5.6 (Trap handler). Special subroutines to handle exception, which can be designed by users.

In the above table, " when trap disabled"means results of operations if trap handlers not used.

Example. $\alpha = 192$, for single precision, $\alpha = 1536$ for double precision. We can't store x since it is too large/small.

1.5.2 Compiler Operations

Compiler may provide a way so the program stops if an exception occurs. For example, SUN's C compiler provides a command

```
-ftrap=t
```

t: `%all, %none, common, [no%]invalid, [no%]overflow, [no%]underflow, [no%]division, [no%]inexact`

Note. common: invalid, division by zero, and overflow

when compile you will see

```
Note: IEEE floating-point exception flags raised:
    Inexact;  Underflow;
See the Numerical Computation Guide, ieee_flags(3M)
```

Note. gcc doesn't have the functionality to explicitly detect exceptions. It only provides

- `-fno-trapping-math`: The default if `-ftrapping-math`. Setting this option may allow faster code if one relies on " non-stop"IEEE arithmetic.
- `-ftrapv`: Generates traps for signed overflow on addition, subtraction, multiplication.

1.5.3 Trap Handlers

For example,

```
do {
...
} while { not x >= 100; }
```

If `x = NaN`, an infinite loop will occur. Any comparison involving `NaN` is wrong. Thus `x >= 100` returns `false`. A trap handler can be installed to abort it.

Another example is that calculate $x_1 \times \cdots \times x_n$ may overflow in the middle, but fine in the end.

```
for (i = 1; i <= n; i++)
    p = p * x[i];
```

If trap handlers are not used, the result will be ∞ and wrong, i.e.

$$x_1 \times \cdots \times x_r, r \leq n \text{ overflow }, \quad x_1 \times \cdots \times x_n \text{ in the range}$$

> **Note.** There might be a solution,
>
> $$e^{\Sigma \log(x_i)}$$
>
> but quite unaccurate and cost more.

A trap handler can be installed to save the intermediate result

```
for (i = 1; i <= n; i++) {
    if (p * x[i] overflow) { // trap handler to catch overflow
        p = p * pow(10, -a);
        count = count + 1 ;
    }
    p = p * x[i];
}
p = p * pow(10, a*count) ;
```